Gaspard. B, Thomas. B, Hugo. D, Noam. GO, Maxence. P

datsmysong

-

Rapport final

Tuteur : Florian Ferbach

Année scolaire : 2023 - 2024

Table des matières

Table des matières 2
Introduction 3
Présentation de l'équipe 3
Planning de déroulement du projet durant les 6 itérations du projet 3
Première itération 4
Deuxième itération 4
Troisième itération 4
Quatrième itération 4
Cinquième itération 5
Sixième itération 5
Analyse 6
Modification de notre planning de déroulement par rapport à l'étude préalable 7
Découpage fonctionnel 9
Quelques diagrammes UML importants 11
Réalisation 12
Intégration de plusieurs plateformes (orienté objet) 12
Communication en temps réel 13
Difficultés rencontrés 14
Analyse et bilan personnel du sujet 15
Thomas 16
Hugo 17
Noam 18
Maxence 19
Gaspard 20
Objectifs atteints à la fin du projet 21
Bilan sur les deux dernières itérations 21
Mise en production 21
Résumé des fonctionnalités 22
Reprise du projet 23
Conclusion 24

Introduction

Ce document est le rapport final du projet tutoré nommé datsmysong tuteuré par Florian Ferbach.

Ce projet consiste à la réalisation d'une application mobile et web dont l'objectif est de permettre une écoute musicale à partir d'une file d'attente de musique collaborative, à l'instar de la fonctionnalité Spotify appelée Spotify Jam[1]. Ce type d'application serait grandement utile lors de rassemblements festifs entre amis ou en famille afin que tout le monde puisse ajouter sa touche à l'ambiance musicale.

L'application doit pouvoir fonctionner avec différentes plateformes musicales, on parlera de « platform agnostic ». L'utilisation ne sera donc pas limitée à Spotify, ni à une autre plateforme.

Les utilisateurs sont regroupés dans des salles d'écoute dans lesquelles on dira qu'ils sont "participants", ces salles d'écoute sont créées par un utilisateur qu'on appellera "hôte", et chaque salle d'écoute comportera une file d'attente partagée avec les musiques que les participants auront ajouté.

Présentation de l'équipe

Nous sommes un groupe de 5 personnes :

- B. Gaspard Chef d'équipe / Développeur / Spécialiste authentification
- B. Thomas Développeur / spécialiste création de salles
- D. Hugo Développeur et responsable communication en temps réel
- G.O. Noam Développeur et testeur iOS
- P. Maxence Chef de projet / Développeur

Planning de déroulement du projet durant les 6 itérations du projet

Voici le résumé des tâches terminées sur la branche main, à chaque fin d'itération

Note : chaque itération comporte uniquement les tâches terminées, il se peut donc qu'un membre du groupe n'apparaisse pas dans le cas où il n'aurait pas fini une tâche assignée.

[1] https://support.spotify.com/fr/article/jam/

Première itération

- Créer un compte utilisateur à partir d'un compte Spotify et se connecter
- Créer une salle d'écoute
- Création et intégration d'un composant Bouton

Deuxième itération

- Créer un compte utilisateur à partir d'un compte Google
- Suppression des bugs liés à l'authentification
- Créer les maquettes nécessaires pour la page des intégrations

Troisième itération

- Générer un lien vers une salle d'écoute et rejoindre une salle d'écoute grâce à ce lien
- Création de compte à partir d'une adresse email et d'un mot de passe
- Modification du composant Bouton afin d'y ajouter une propriété de couleur

Quatrième itération

- Créer la logique des interactions avec les plateformes de musique
- Implémenter la logique pour chaque plateforme de streaming
- Menu d'intégrations des plateformes (Spotify et SoundCloud implémenté)
- Modifier les paramètres de son profil (nom, email, avatar stocké en base de donnée)
- Fermeture d'une salle d'écoute pour le créateur de la salle
- Suppression d'une musique pour le créateur de la salle
- Ajouter une musique à la file d'attente (backend)
- Consulter la file d'attente d'une salle d'écoute
- Afficher un QR code sur un écran de partage
- Rejoindre une salle d'écoute à l'aide d'un code

Cinquième itération

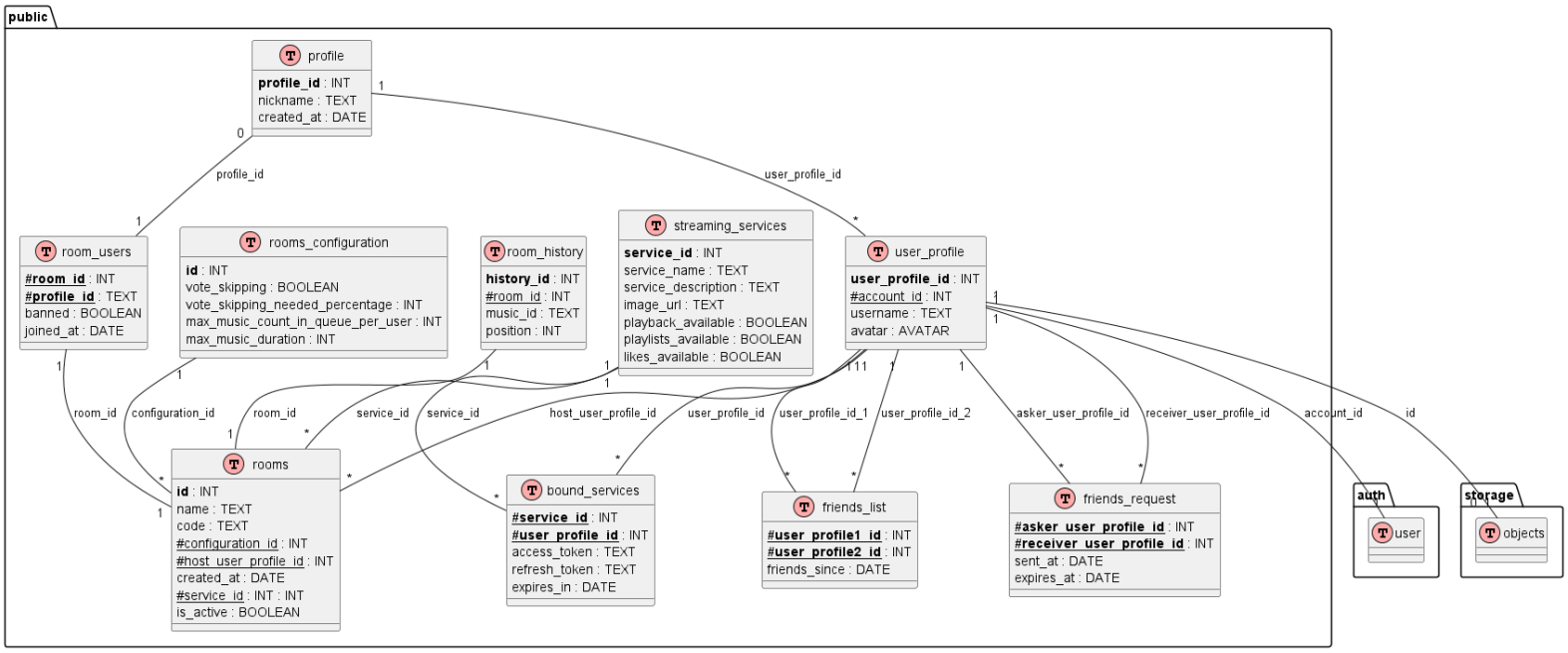
- Quitter une salle d'écoute en tant que participant
- Ajouter une musique via une barre de recherche
- Améliorer le middleware d'authentification pour la création de sessions

Sixième itération

- Améliorer la robustesse du front
- Régler un bug qui arrive sur la production lors de la connexion (la popup ne se ferme pas une fois l'authentification terminée)
- Mettre à jour la bottom navigation en suivant les maquettes
- Gestion de la musique en cours de lecture
- Créer le menu de paramétrage d'une salle d'écoute
- Voter pour passer une musique (en cours, ou en file d'attente)
- Intégrer les maquettes pour l'onglet "Profil"
- Intégrer les icônes de Phosphor Icons dans l'application
- Mettre à jour la police en suivant les maquettes
- Intégrer les maquettes pour l'onglet "Amis"
- Consulter l’historique et les détails d’une salle d’écoute passée

Analyse

Notre base de données a subi de légères modifications par rapport au diagramme initial que nous avions imaginé. Nous avions lors de l'itération 1 créé une nouvelle table active_rooms, partagent en grande majorité le schéma de la table rooms, permettant ainsi de consulter les salles actives et inactives dans deux tables différentes, mais nous sommes revenu sur ce choix qui nous a forcé à dupliquer des données et a surtout complexifié notre schéma et par conséquent notre logique métier avec au bout du compte très peu de gain. Nous avons donc pris la décision lors de l'itération 4 de rassembler ces deux tables à nouveau.



Modification de notre planning de déroulement par rapport à l'étude préalable

Itération 1

- Créer un compte (seulement compte Spotify) (Gaspard)
- Création d'une salle d'écoute ~~(Maxence)~~ Thomas
- Générer un lien vers la salle d'écoute et rejoindre une salle d'écoute grâce à un lien (Noam)
- Ajout d'une musique possible uniquement avec un lien Spotify ~~(Thomas)~~ Hugo
- Consulter la file d'attente (Hugo)
- Créer la logique frontend des interactions avec les plateformes de musique (qui dépendent de la nature de la plateforme de streaming à intégrer, API ou Widget par exemple) (Maxence)
  - Créer le schéma de données à utiliser côté frontend

Itération 2

- Implémenter pour chaque plateforme de streaming les méthodes du lecteur audio (volume, play/pause, suivant/précédent, ajout en file d'attente)
  - Spotify ~~(Gaspard)~~
  - SoundCloud (Maxence)
- Créer un compte (adresse mail et mot de passe) ~~(Hugo)~~ Gaspard
- Lier son compte Spotify (Thomas)
- Consulter l'historique et les détails d'une salle d'écoute (participants, musiques jouées...) ~~(Noam)~~ Maxence

+ Rejoindre une salle d'écoute (page rejoindre) (Noam)

+ Authentification avec Google (Gaspard)

Itération 3

- Ajouter une musique (barre de recherche, lien d'autres plateformes...) (Maxence)
- Voter pour passer une musique (Gaspard)
- Supprimer une musique de la file d'attente (Noam)
- Paramétrer une salle d'écoute (vote, nombre de musiques max par utilisateur...) (Thomas ~~& Hugo~~)

Itération 4

- Rejoindre une salle d'écoute avec un petit code ou un QR code ~~(Gaspard)~~ Noam
- Fermer une salle d'écoute (hôte) (Hugo)
- Quitter une salle d'écoute (participant) (Noam)

+ Créer le menu de paramétrage d'une salle d'écoute (Thomas)

Itération 5

- ~~Expulser/bannir un participant de la salle d'écoute (Thomas)~~
- ~~Liaison du compte avec services populaires (Deezer, etc...) (Gaspard)~~
- ~~Permettre à un participant d'accéder à ses playlists et coups de coeurs sur les plateformes de streaming liées (Spotify, Deezer, etc...) et d'ajouter ces musiques là dans la salle d'écoute (Maxence)~~

Commencé itération 3

Finalisé itération 4

+ Gestion de la musique en cours de lecture

Itération 6

- ~~Rejoindre une salle d'écoute en tant qu'invité (via un lien) (Noam)~~
- ~~Système d'amis (consulter, supprimer, demander, refuser) (Hugo)~~
- Modifier les paramètres de son profil (nom d'utilisateur, avatar, etc) ~~(Thomas)~~ Gaspard

+ Améliorer la robustesse du front (Gaspard)

+ Réglage de bug sur la production pour l'authentification (Gaspard)

+ Mettre à jour la bottom navigation en suivant les maquettes (Noam)

+ Ajout de lidentifiant de l'utilisateur ayant ajouté la musique en file d'attente (Noam)

+ Ajouter la librairie d'icon dans l'application(Gaspard)

+ Mettre a jour la police en suivant les maquettes (Noam)

+ Mettre une indication pour les pages non faite (amis) (Gaspard)

+ Ajout des règles de sécurité sur notre base de donnée (Gaspard)

+ Intégrer les maquettes pour les onglets Profil (Gaspard), Acceuil (Maxence), Salles (Thomas)

+ Demande d'un nom unique lors de la création d'un compte à l'aide d'un provider (Gaspard)

Tâche déjà commencée, et terminée sur l'itération pointée

Tâche commencée et terminée sur l'itération pointée

Tâche déplacée

Tâche commencée lors de l'itération, mais non terminée

Tâche commencée et terminée durant l'itération

Tâche non commencé durant l'itération

+ lorem.. Ajout d'une nouvelle tâche non prévue (redécoupage / oubli)

Tout au long de nos itérations, notre planning préalable n'a pas vraiment été respecté. Nous avions réalisé une liste de tâche dans un ordre précis pour le développement de notre application, mais nous avons dû les assigner à des itérations en n'ayant aucune idée de la durée de chacune de celle-ci, variable sur des imprévus ainsi que l'aisance de la personne à qui la tâche est assigné. De nombreuses tâches ont simplement été commencées en cours d'itération, et ainsi terminé sur l'itération suivante. Ce sont les tâches encadrées en orange. Mais nous avons aussi eu un grand nombre de tâches ajoutées lors de la dernière itération.

En effet, nous avons pu lister tous les problèmes que notre application avait quelques semaines avant la fin du projet, et nous avons pu prendre les bonnes décisions pour trier, par importance, les tâches restantes, avec celles ajoutées. C'est pourquoi trois tâches de l'itération 5 et 2 de l'itération six ont été supprimées. Nous n'avons pas eu le temps de faire le système d'amis, le système de rejoindre en tant qu'invité donc sans compte, ou même lier d'autre service ne permettant pas le streaming de musique, mais de relier tout de même notre application à la bibliothèque musicale.

Nous pouvons souligner toutefois que, bien que le schéma soit décousu, notre organisation n'en était pas tant. Nous avons utilisé, comme Trello, un système de Kanban pour chaque itération, mais sur GitHub directement, ce qui nous a permis de nombreuses intégrations très pratiques pour chaque tâche, d'avoir le code ajouté à notre branche main, de pouvoir faire des relectures de code etc de manière extrêmement organisé. Donc le fait que les tâches se soient commencées plus tard, aient pris plus de temps où autre n'a posé aucun souci

d'organisation. En plus de nous avoir fait gagner du temps, cela nous a permis aussi d'acquérir une bonne maîtrise d'un outil extrêmement utilisé dans le milieu de l'informatique.

Découpage fonctionnel

Chaque personne du groupe choisit ses tâches en fonction de ses appétences et compétences. Sans suivre une stratégie précise, nous avons suivi une certaine logique dans la succession des tâches, nous nous sommes spécialisés, et nous allons vous le présenter tour à tour :

- Maxence : Dès la première itération, Maxence a pris la très lourde tâche de la lecture de musique. C'était l'une des tâches les plus incertaines. En effet, il était compliqué de déterminer la durée de cette tâche, nous n'avions pas une idée certaine de l'implémentation et nous l'avons donc commencé dès la première itération. Elle s'est avérée assez longue, quasiment quatre itérations. Par la suite, Maxence à continuer par la tâche de relier la file d'attente stockée sur notre backend à notre lecteur côté front, il a donc bien travaillé avec toutes les technologies, autant le backend que le serveur, en passant par la base de données pour y enregistrer les musiques déjà passées.

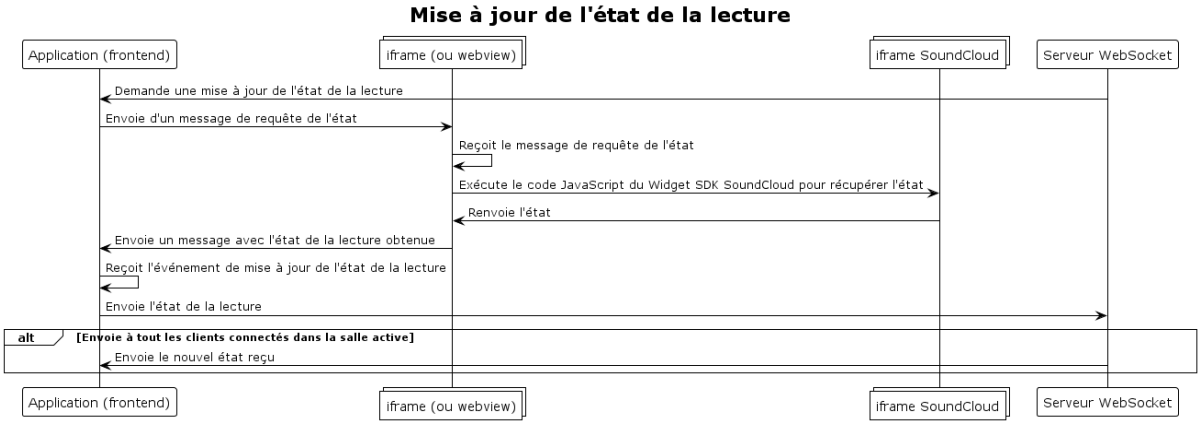
- Gaspard : Dans le concept de notre application, nous devions avoir accès à des informations des comptes au moins sur Spotify de nos utilisateurs. C'est pourquoi nous nous sommes penchés dès le début sur l'authentification à deux facteurs, car elle nous permettait de récupérer les données nécessaires à faire les requêtes sur le compte des utilisateurs. Gaspard s'est occupé de l'authentification, avec mot de passe, avec Google, ainsi qu'avec Spotify. Ainsi que la création de compte. Il a réalisé d'autres tâches, mais s'est spécialisé sur l'authentification sur les premières itérations.

- Hugo : Pendant que tous les membres s'occupaient d'ajouter des pages à notre application, Hugo a développé une architecture dans notre backend nous permettant, en orienté objet, de pouvoir implémenter les différentes plateformes de manière agnostique. Il a su intégrer Spotify, contacter leur API pour réaliser un objet dans notre format, et le fournir sur le site web. Pour cela, il s'est attelé à intégrer les Websockets, qui ont été utilisés par d'autres membres du groupe par la suite pour d'autres fonctionnalités.

- Thomas : Les salles d'écoutes sont le cœur de notre application, et c'est Thomas qui a implémenté ces pages. Que ce soit le menu de paramétrage des salles, de création, la consultation de l'historique d'une salle ou même simplement l'affichage de la salle active, c'est lui qui a fait la partie front. De plus, c'est un développeur full-stack. Il a aussi des routes sur notre backend, notamment pour l'intégration de SoundCloud.

- Noam : Les entrées/sorties et invitations des participants sont un point important des salles d'écoutes, car c'est ce qui fait qu'organiser des fêtes avec notre application est chose aisée. La majorité des tâches en lien avec ce point ont donc été confiées à Noam. Il a pu implémenter l'invitation aux salles d'écoute sous différentes formes, ainsi que le comportement lorsque quelqu'un rejoint et quitte une salle. Il a également pu prendre part au projet en tant que testeur iOS. C'était un point essentiel si l'on voulait une application mobile fonctionnelle sur toutes les plateformes, et il était le seul possédant un appareil sous ce système d'exploitation, le rôle lui a donc automatiquement été attribué.

Quelques diagrammes UML importants



Ce diagramme de séquence démontre la communication entre le frontend, le backend (serveur websocket) ainsi que l'iframe chargé au niveau de l'app frontend, au moment de récupérer l'état de la lecture. C'est ce modèle de communication qui est utilisé pour les salles d'écoute dont la lecture se déroule client-side, c'est-à-dire, côté client, comme par exemple avec le widget SoundCloud.

Réalisation

Nous avons une architecture client/serveur relativement classique, avec un backend TypeScript utilisant le framework Fastify (alternative plus performante et légère à ExpressJS), et un client React Native utilisant le framework Expo et de nombreux modules.
En ce qui concerne notre base de données, nous utilisons le service Supabase, qui héberge une base de données PostgreSQL, fournit un SDK TypeScript avec une API moderne et dont l'expérience développeur est inégalée, ce qui nous permet de rapidement avancer.
Nous n'avons pas de test E2E ou unitaires, car créer des tests sur un projet Web (et qui plus est, natif) est une tâche lourde demandant énormément de temps, ce qui n'était pas compatible avec les contraintes de temps que nous avons.
En revanche, nous avons des configurations Prettier et ESLint permettant d'obtenir des avertissements voire des erreurs dès que la qualité du code n'atteint pas les règles définies, et des pipelines CI vérifiant le bon respect de ces règles à chaque PR ouverte, permettant ainsi de s'assurer de la bonne qualité du code.

Intégration de plusieurs plateformes (orienté objet)

L'un des plus gros enjeux de notre projet était l'intégration de plusieurs plateformes musicales. Lors de l'étude préalable, nous avions analysé les 5 plateformes musicales connues afin de nous concentrer sur celles permettant une intégration de notre application. Spotify et Apple Music étaient les deux plateformes permettant une intégration de notre application avec le lecteur appareil de l'utilisateur, mais Apple Music étant payant, nous n'avons pas pu le retenir. Nous devions donc trouver une autre plateforme afin de commencer avec 2 plateformes différentes. Nous nous sommes donc intéressés à SoundCloud, qui ne possède pas d'API pour gérer la lecture sur le compte de l'utilisateur, mais qui nous laisse la possibilité de créer un lecteur sur l'une de notre page Web à l'aide d'un Widget.
Pour résumer, nous avons déterminé deux types de plateformes pour la lecture de notre musique :

- lecteur de musique locale
- lecteur de musique distante

L'enjeu était de se concentrer sur une implémentation réfléchie pour que les deux types de lecture soient gérés de la même manière côté client au niveau des contrôles du lecteur (pause, play, suivant, etc), et que nous gardions la possibilité d'ajouter d'autres services par la suite tels qu'Apple Music, sans devoir modifier toute l'architecture de notre application.

En lien avec la partie d'après sur les Websocket, nous devions implémenter une communication en temps réel entre les utilisateurs dans une salle active pour que l'état de la lecture ainsi que la file d'attente par exemple soient mis à jour pour chacun dès que nécessaire, c'est-à-dire en temps réel. Le moyen le plus simple est de passer par un serveur central. Afin d'implémenter de la même manière le Websocket que l'on soit sur un lecteur de musique local ou distant, nous avons utilisé une approche orientée objet en TypeScript, avec une interface étant implémentée par chaque service et le serveur pourra simplement gérer en appelant sur un objet du type de notre interface.

Communication en temps réel

Lors de notre analyse, nous avions déclaré comme risque la communication en temps réel et la synchronisation entre les différents appareils. C'est un élément fondamental de notre application et dès la première itération, Hugo s'est lancé dans cette tâche. Cette tâche a pris quasiment 3 itérations à être terminée, car elle était assez complexe, et nous avons pris le temps de faire une bonne architecture pour la communication en temps réel. En effet, la tâche d'Hugo ne comportait que consulter la liste de lecture, mais les WebSocket n'étant pas encore implémentés dans l'application, c'est à lui qu'est revenue la tâche de réaliser l'infrastructure de code nécessaire pour les supporter. Par la suite, Maxence a dû attendre une itération complète en mettant en pause ses deux premières tâches, nécessitant cette communication en temps réel pour avancer.

Difficultés rencontrés

La difficulté que nous avons tous éprouvée est le ralentissement lié aux technologies utilisées. En effet, tous les membres n'étaient pas au même niveau de familiarité avec les outils que nous avons décidé d'utiliser. Pour surmonter cet obstacle, nous avons mis en place une stratégie efficace en assignant des tâches en fonction des compétences individuelles et de l'appétence de chacun envers chacune des tâches. Cette approche a permis de maximiser l'efficacité du groupe, car chacun pouvait contribuer de manière significative en fonction de ses connaissances et de son expérience. Malgré les difficultés initiales, la découverte de ces technologies innovantes nous a non seulement permis de renforcer nos compétences individuelles, mais aussi de créer un environnement d'apprentissage collaboratif au sein de l'équipe. Nous avons dès le début instauré des *code review* où chacun devait consulter et commenter le code des autres, dans le but de lever de potentiels problèmes et d'élever la qualité du code. A partir de la 3ième itération, nous avons appliqué notre nouveau workflow suggéré par notre tuteur, qui contenait une heure de code review dédiée à partir de 17h, permettant ainsi de consulter le code écrit la journée par nos camarades.

Analyse et bilan personnel du sujet

Nous allons, à tour de rôle, présenter notre bilan personnel du projet, ce qu'il nous a apporté, ce que nous avons découvert (technologie, méthode de travail, travail de groupe), et sur les compétences acquises.

Thomas

Je pense personnellement que j'étais la personne du groupe avec le moins de compétences techniques. Je n'avais jamais touché aux technologies que nous avons utilisées, que ce soit react et encore moins react-native. Je n'avais non plus jamais utilisé d'API REST, comme fastify. La seule seule chose que je connaissais un minimum, c'était supabase, car nous l'avions utilisé durant le cours de développement mobile en S4.

Je pense qu'à la fin de ce projet tutoré, je commence à me sentir vraiment à l'aise avec react-native et fastify. Cela est grandement dû aux review faîtes par mes coéquipiers qui avait plus de compétences que moi dans ces technologies. D'ailleurs, cela me permet aussi de rebondir sur un autre point, sur lequel je pense avoir grandement appris : GitHub. En effet, GitHub est très complexe, et ça je le savais bien avant d'avoir commencé ce projet tutoré. Néanmoins, durant ce projet, c'était la première fois que j'avais l'impression d'utiliser git professionnellement, entre les branches avec une convention de nommage bien précise, pareil pour les messages de commit. C'était aussi la première fois que je faisais des pull request, et des reviews des pull request des autres personnes de mon groupes. Je me sens aussi beaucoup à l'aise avec les conflits que l'on peut rencontrer durant les rebases. Pour résumer, je pense vraiment m'être grandement amélioré avec Git et je suis bien plus à l'aise.

Un des derniers points aussi qui m'a grandement plus, et qui m'a réellement donné l'impression d'être dans un milieu professionnel, est la mise en place d'un plan journalier. Cela nous avait été conseillé par notre tuteur M. Ferbach, quand je parle d'un plan journalier, je veux parler de la mise en place de réunion journalière le matin, et chaque jours à 17h, nous faisions des reviews des pull request de nos coéquipiers. Les réunions journalières me permettaient de me fixer un objectif pour la fin de la journée, ce qui me motivait plus. Pour ce qui est des reviews, voir comment code les autres personnes est aussi très bénéfique, et me permet de me remettre en question sur ma façon de coder.

Pour conclure, je pense que j'ai grandement appris au niveau technique, donc au niveau des nouveaux langages que j'ai appris, ainsi que Git. Mais je pense aussi avoir beaucoup appris au niveau des méthodes de travail, ce qui je pense est tout autant intéressant que la technique.

Hugo

J'ai pu expérimenter et mettre en place de nombreuses technologies dont j'avais déjà entendu parler sur internet mais dont je n'ai jamais fait l'usage comme React et en particulier React Native.
En effet, le domaine des framework web était assez étranger pour moi, qui était habitué à faire mes sites web en vanilla (nb en français, nature), c'est-à-dire avec uniquement du html, css, et javascript.
Ici, nous avons utilisez React Native, afin de voir large et de permettre à terme une exportation facile vers des applications native, comme son nom l'indique, vers les principales plateformes mobile : Android et iOS, même pour l'instant l'utilisation en tant que site web convient très bien pour nos tests mais aussi pour l'utilisation en production.
La différence vient des balises utilisées : React utilise des balises html alors que React Native utilise ces propres balises propriétaires, mais les techniques de programmation sont quelque peu équivalentes notamment la notion de composant qui regroupe un ensemble d'élément de base mis en contexte avec la possibilité d'y passer des paramètres.

j'ai été aussi le premier du groupe à mettre en place, la communication en temps réel entre le client, le site web ou application et serveur, gérant le stockage des données, la liaison avec les multiples clients, et l'authentification. Ma vision d'architecte logiciel en est sortie renforcée, car la transformation d'un modèle de communication unilatérale, du client vers le serveur, a dû être adaptée à la communication bilatérale permettant plus de souplesse dans la conception de notre application.

Noam

Cela a été mon plus grand projet jusqu'à maintenant, sur lequel j'ai pu apprendre beaucoup de choses, que ce soit au niveau technique ou personnel, et j'ai également pu découvrir ce que je valais vraiment, ainsi que ce que je voudrais reproduire et intégrer ou non dans mes futurs projets.

Tout d'abord, sur le point technique, j'ai pu expérimenter de nombreuses choses, étant donné que c'était la première fois que je développais une vraie application en utilisant ces technologies. Cela a été très enrichissant, j'ai pu apprendre les bonnes et mauvaises pratiques en tant que développeur web et mobile et j'ai acquis quelques notions en UI et UX. Ayant participé au projet principalement en tant que développeur frontend, j'ai pu grandement me familiariser avec Expo et React-Native, et je suis désormais beaucoup plus à l'aise qu'au tout début du projet. J'ai pu au passage me remémorer le CSS appris en première année, et apprendre de nouvelles choses. Quant au backend, j'ai pu comprendre le fonctionnement général du WebSocket, même s'il reste des choses à revoir. J'ai également pu me réconcilier avec GitHub, avec lequel j'avais beaucoup de mal au début de ce projet, et dont j'ai appris de nouvelles facettes. Les méthodes de travail, elles, m'étaient déjà familières grâce au stage de deuxième année, mais cela nous a permis de remonter la pente et de travailler efficacement au cours des itérations suivantes. Le travail de groupe n'a cependant pas été priorisé je pense et c'est ce qui nous a fait défaut, notamment au début du projet. Je pense que c'est un point que j'aimerais mettre au centre lors des prochains projets auxquels je participe, car c'est pour moi la première étape d'un travail bien mené.

J'ai tout de même beaucoup appris à travers ce projet, et cela a été une expérience très enrichissante. J'ai pu devenir un meilleur développeur qu'au début de ce projet, que cela soit à travers mon efficacité, mon autonomie ou mes compétences, et je suis devenu beaucoup plus indépendant qu'au début du projet.

Maxence

Bien que les débuts du projet aient pu être difficiles, notamment au niveau de l'organisation, et que mon investissement dans la partie développement durant les deux premières itérations a été relativement faible, car éclipsé par mon investissement dans la partie gestion de projet, les choses se sont bien améliorées depuis.

J'ai pu penser, concevoir et réaliser une application mobile et web multiplateforme en équipe et ce, avec un volume horaire conséquent ayant permis à chacun de développer ses compétences sur un véritable projet d'envergure.

L'architecture de notre projet est complète et très similaire à ce qu'on pourrait retrouver en entreprise, ce qui rend notre projet véritablement concret.

Avec notre frontend (Expo / React Native), notre backend (Fastify & Socket.IO), notre base de données (PostgreSQL / Supabase), ainsi que toute la partie Ops (CI/CD automatisant le déploiement du frontend, du backend ainsi que la construction d'un APK), nous avons mis en place une infrastructure solide et évolutive pour notre application, en évitant de faire les choses à moitié.

Au global, on peut dire que j'ai pris un rôle de chef de projet puisque c'est moi qui ait dirigé le projet vers ses objectifs, en mettant au clair les attentes des différentes tâches, en m'assurant que les membres du groupe respectaient une certaine qualité de travail en incluant très tôt dans le projet eslint, un outil permettant de linter notre code, c'est-à-dire identifier les points faibles en matière de qualité, sécurité, etc, pour ainsi encourager le développeur à les régler. Cet outil, en combinaison avec un système strict sur GitHub (Pull Request obligatoire pour push dans main, 2 reviews minimum qui approuvent la Pull Request pour pouvoir la fusionner) nous a permis de maintenir une bonne qualité de code tout au long du projet, en évitant les gros conflits ou problèmes lors de la mise en production notamment.

Enfin, bien que j'avais déjà quelques expériences avec React dans le développement web, l'utilisation d'Expo m'a permis d'apprendre le développement sous React Native, et donc la création d'applications mobiles avec ce framework. Cela a naturellement permis d'étoffer mes compétences, et je suis très content du résultat que l'on obtient à la fin de cette sixième et dernière itération.

Gaspard

Ce fut le plus gros projet auquel j'ai participé, et la première fois que j'ai pu prendre quelques responsabilités dans un groupe.

Techniquement, je faisais partie des deux personnes maîtrisant le mieux les technologies abordées. En effet, j'ai pu utiliser sur différents projets Supabase, ainsi que React. Ce n'étaient pas les premières lignes de TypeScript que j'ai pu inscrire et c'est pourquoi nous avons pu mettre en place différentes bonnes pratiques dès le début du projet. Le plus important selon moi, ce qui a permis au projet de bien se passer, a été la place et la gestion de GitHub dans le groupe. Grâce à notre tuteur et de la bonne volonté pour apprendre et respecter différentes règles, nous n'avons pas eu de soucis ou conflits, de groupes vis-à-vis de cet outil de collaboration. Le système de branche, PR, rebase pour récupérer les changements actifs, a été maîtrisé par tout le monde assez tôt, et c'est un élément précurseur pour que tout le monde réussisse à s'épanouir dans le projet.

C'est aussi grâce à notre utilisation de GitHub que nous avons pu collaborer et améliorer, dans un premier temps, notre qualité de code. En effet, régulièrement, nous avons dû relire le code de camarades pour débattre d'amélioration ou de meilleures méthodes à mettre en place. Ce n'est pas une chose qui est suivie dans tous les projets, mais M. Florian Ferbach nous a tout de même fortement incité à le faire de façon régulière (lors des semaines entières consacrées au projet, nous avions convenu d'une heure de relecture de code sur une journée de 8 h). C'est une implication de plus dans le projet, qui nous permet à tous de collaborer pour s'améliorer, et c'est, pour moi, une réussite de notre projet.

Nous sommes allés moins vite que ce que nous avions prévu lors de l'étude préalable, mais nous avons su inscrire un code robuste, réutilisable, compréhensible, compétence attendue d'un développeur. Par ailleurs, nous avons été "félicités" de cette qualité par notre tuteur par une quasi-note parfaite sur ce point.

Nous avons perdu du temps sur d'autres points bien entendu, notre découpage des tâches était critiquable, nous avons eu des tâches bien trop longues sur lesquelles nous aurions dû rebondir en les re-découpant, en travaillant plus en groupe, mais nos méthodes de travail nous ont permis pour chacun de choisir les tâches par appétences et d'apprendre.

Objectifs atteints à la fin du projet

Bilan sur les deux dernières itérations

Les deux dernières itérations sur les six réalisées ont été les plus productives. En effet, nous avons pu délivrer de nombreuses tâches. Cela s'explique par le fait que nous ayons gagné en efficacité sur les technologies et les méthodes de travail. Nous avons eu aussi des tâches supplémentaires pour un premier lieu, la mise en production du projet, mais aussi pour corriger tous les petits problèmes que nous avons relevés lors de l'itération.

Mise en production

La finalité d'un projet, bien que toutes les tâches planifiées n'aient pas été réalisées, est tout de même, pour nous, de produire un livrable. Il est important de rappeler que nous utilisons Expo, une librairie JavaScript open-source permettant de réaliser une application mobile native sous Android, iOS ainsi qu'un site web. Tout au long de notre projet, en testant à différent moment notre application, nous avons pu voir de nombreuses différences visuelles, mais aussi fonctionnelles entre les différentes plateformes. C'est l'un des facteurs de ralentissement, c'est une difficulté connue dans le développement d'applications sur plusieurs plateformes avec une seule base de code commune. Mais nous avons tout de même pris le temps pour n'oublier aucune plateforme. Hugo a pu réaliser un workflow GitHub Actions générant l'APK (Android) à chaque nouvelle modification sur le main, mais cet APK n'est pas mis en ligne sur le Play Store puisqu'une licence est requise. Nous avons malgré tout pu le tester sur nos appareils. Produire un workflow similaire pour iOS était impossible pour nous, puisque cela requiert tout d'abord la licence Apple Developper, mais aussi et surtout la possession d'un MacBook de la marque.
Cependant, nous nous sommes intéressés à déployer sur la plateforme la plus accessible : le web. En effet, nous avions intérêt, et assez tôt dans le projet, d'automatiser le déploiement du site, afin de nous assurer de son bon fonctionnement. Rappelons que nous avons 3 "services" pour que notre application fonctionne. Les deux premiers services sont utilisés quelque soit la plateforme, il s'agit de la base de données ainsi que notre backend. Notre base de données est

hébergée par Supabase directement, cependant nous hébergeons nous-mêmes sur un serveur notre backend. Nous avons à chaque mise à jour de la branche main, un workflow GitHub Actions qui s'occupe de mettre à jour le conteneur Docker sur la machine avant de le redémarrer.
Pour finir, nous hébergeons le site web à l'aide de Vercel, accessible à l'aide d'un nom de domaine acheté pour l'occasion : datsmysong.app.

Résumé des fonctionnalités

Voici le détail des fonctionnalités réalisées, ainsi que celles prévues mais abandonnées. Tout est détaillé sur notre GitHub Project, mais nous pouvons nous rendre compte avec ce résumé que nous avons dû abandonner 2 tâches : le système d'amis et l'utilisation de l'application en tant qu'anonyme.
Nous avons préféré approfondir d'autres fonctionnalités que de se lancer dans une tâche impossible à terminer de manière propre avant la fin du projet.

Fonctionnalités	**Fait**	**Abandonné**
Système d'amis		❌
Lecture de la musique	✅	
Création de compte	✅	
Gestion salle d'écoute	✅	
Ajouts de musique avec recherche	✅	
Rejoindre en tant qu'anonyme une salle		❌
Mise en production	✅	
Gérer son compte	✅	
Consulter une salle passée	✅	

Reprise du projet

Ayant pour volonté de continuer le travail sur ce projet dans le cadre personnel, nous ne souhaitons pas que ce projet soit repris par de prochains élèves. De plus, faire reprendre notre application par de prochains étudiants semble une tâche compliquée au vu des technologies que nous avons utilisées, qui leur seront probablement inconnues.

Conclusion

Durant ces quatre mois, nous avons, nous 5, appris énormément, aussi bien d'un point de vue compétences techniques, que soft skills, notamment en travail en équipe. Le volume horaire conséquent et la durée du projet ont permis à chacun de s'investir pleinement.
Malgré des difficultés d'organisation durant les premières itérations, liées notamment aux grandes différences de connaissance des technologies utilisées, nous avons su au cours des itérations s'organiser de mieux en mieux afin d'augmenter notre vélocité.
Notre workflow mis en place par Florian Ferbach au cours de l'itération 3 nous a permis de prendre connaissance du code écrit par chacun, et donc au fur et à mesure d'avoir une meilleure vue d'ensemble de l'application, permettant ainsi de gagner du temps au moment d'implémenter une nouvelle fonctionnalité nécessitant une bonne connaissance de l'architecture logicielle que nous avons développé.
Nonobstant certaines tâches abandonnées lors de la dernière itération par manque de temps, nous sommes fier du travail accompli, car une grande majorité de fonctionnalités ont pu être développés à temps, et grâce aux bonnes pratiques mises en œuvre depuis le début du projet, cette grande majorité de fonctionnalités n'ont pas été implémentés entre deux portes.
Notre architecture logicielle, ainsi que ce qui tourne autour, notamment niveau opérationnel (automatisations avec GitHub Actions) ont permis à chacun de développer dans les meilleures conditions, détecter les problèmes avant la fusion dans la version finale de l'application, maintenir une bonne qualité de code, et tester le plus tôt possible les nouvelles fonctionnalités.

Au fil des itérations, nous avons chacun forgé ce que datsmysong est aujourd'hui. Cela n'a pas toujours été facile, puisque nous avons tous des personnalités différentes qu'il faut accorder pour avancer en tant que groupe. Aujourd'hui, nous sommes fiers du travail accompli, et des compétences que nous avons tous développées.

Pour finir, nous aimerions remercier notre tuteur, Florian Ferbach, pour son accompagnement précieux tout au long du projet.